The Feast of Divine Judgement

Abstract

Food image classification is a multiclass problem that involves identifying food categories from visual information. This project, The Feast of Divine Judgement, builds a deep learning classifier on the Food-11 dataset, which has eleven food categories. It covers the full workflow of preprocessing, model development, hyperparameter optimization, evaluation, and application development.

The model uses transfer learning with MobileNetV2 pretrained on ImageNet, a custom classification head, and selective fine-tuning. Images are resized to 256 × 256 pixels and augmented with random flipping, rotation, brightness and contrast adjustments, zooming, and random erasing. Hyperparameters (learning rate, batch size, dropout rate, epochs, L2 regularization, and number of fine-tuned layers) are tuned with scikit-learn's ParameterSampler, a randomized search that costs less than an exhaustive grid search. Validation macro F1-score is the selection metric.

The final model reached a test accuracy of 89.90%, macro precision of 89.55%, macro recall of 91.85%, and macro F1-score of 90.43%, with a test loss of 0.3316. A Streamlit web application, with MongoDB integration for information on each predicted food category, lets users upload a food image and see the predicted category with its confidence score.

1. Introduction

1.1 Background

Food image classification is a computer vision task that involves identifying food categories from digital images. It has applications in automated food recognition, dietary monitoring, food-related information systems, and intelligent applications that process visual data. However, accurately classifying food images can be challenging because different food categories may share similar visual characteristics, while images belonging to the same category can vary significantly in appearance, texture, colour, and presentation.

Traditional machine learning approaches often require manually designed features to represent image characteristics. Deep learning, particularly convolutional neural networks (CNNs), enables models to learn hierarchical visual representations directly from image data. Nevertheless, training deep neural networks from scratch can require substantial computational resources and large quantities of labelled training data.

Transfer learning addresses this challenge by reusing knowledge acquired by a model trained on a large dataset [1]. In this project, MobileNetV2 pretrained on ImageNet is employed as the foundation for a food image classification model. The pretrained network is adapted to classify images into the eleven categories represented in the Food-11 dataset.

1.2. Problem statement

The objective of this project is to develop a deep learning-based system capable of classifying food images into their respective categories. The system must learn discriminative visual features while accounting for differences in the number of training samples available across classes.

The project also addresses the challenge of selecting an effective model configuration. Parameters such as learning rate, batch size, dropout rate, regularization strength, and the number of fine-tuned layers can influence classification performance and generalization. Therefore, a systematic hyperparameter optimization process is required to identify a suitable configuration.

Beyond model development, the project aims to make the classifier accessible through a web-based interface that accepts an uploaded image and displays the predicted food category and its confidence score. Database integration is used to retrieve additional information associated with the predicted category.

1.3. Objectives

The primary objectives of *The Feast of Divine Judgement* are:

1. To develop a multiclass food image classification model using the Food-11 dataset, which contains eleven food categories.
2. To preprocess images and apply data augmentation techniques to improve the diversity of training samples.
3. To implement transfer learning using MobileNetV2 pretrained on ImageNet, with a custom classification output layer and selective fine-tuning.
4. To perform randomized hyperparameter optimization using scikit-learn's ParameterSampler.
5. To select the best model configuration using validation macro F1-score as the primary selection metric.
6. To evaluate the final model on the held-out test dataset using accuracy, macro precision, macro recall, macro F1-score, and test loss.
7. To develop a Streamlit-based application for image uploads and food category predictions.
8. To integrate MongoDB for retrieving additional information about the predicted food category.

1.4. Scope of the project

The project focuses on supervised multiclass image classification using the Food-11 dataset [2]. The dataset is organized into training, validation, and evaluation subsets, allowing model training, hyperparameter selection, and final testing to be conducted separately.

The modelling component uses MobileNetV2 transfer learning, image preprocessing, data augmentation, regularization, and selective fine-tuning. Hyperparameter optimization is conducted using randomized sampling, with validation macro F1-score guiding model selection. The selected model is subsequently evaluated on the held-out test set.

The application component uses Streamlit to provide an interface for uploading food images and displaying predictions. MongoDB stores category-related information that can be retrieved after a prediction is generated.

The system is limited to the eleven categories represented in the dataset. It is not designed to identify arbitrary food items outside these categories, estimate nutritional values, determine portion sizes, or provide dietary recommendations. Its performance may also be affected by image quality, unfamiliar food presentations, and class imbalance in the available training data.

1.5. Organization of the report

This report is organized into chapters covering the development and evaluation of the food image classification system. Chapter 1 introduces the project, its motivation, problem statement, objectives, and scope. Chapter 2 presents the technical background and relevant concepts underlying the proposed approach, including convolutional neural networks, transfer learning and MobileNetV2.

Chapter 3 describes the Food-11 dataset and presents the exploratory data analysis. Chapter 4 explains the image preprocessing pipeline and data augmentation techniques. Chapter 5 discusses the model architecture and training methodology, while Chapter 6 describes the hyperparameter optimization and model selection process.

Chapter 7 presents the experimental results and evaluates the final model on the test dataset. Chapter 8 describes the Streamlit application and its integration with MongoDB. Finally, Chapter 9 examines the limitations of the system and potential future improvements, followed by Chapter 10, which concludes the project.

2. Technical background and related work

2.1. Image classification

Image classification is a supervised learning task in which an input image is assigned to one of a predefined set of classes. A classification model learns patterns from labelled training images and uses the learned representations to predict the class of previously unseen images.

In food image classification, these patterns may include colour, texture, shape, and the spatial arrangement of visual elements. However, classification can be challenging when different food categories exhibit similar visual features or when images from the same category differ considerably in appearance.

The Food-11 classification task consists of eleven distinct categories. Consequently, the model must learn features that distinguish these categories and produce a probability distribution over the possible classes for each input image.

2.2. Convolutional Neural Networks (CNN)

Convolutional neural networks (CNNs) are deep learning models widely used for image-related tasks. They process images through layers that learn increasingly complex visual representations. Early layers typically identify simple patterns, such as edges and textures, while deeper layers learn more complex combinations of features.

A typical CNN consists of convolutional layers, activation functions, pooling or downsampling operations, and classification layers. Convolutional layers apply learnable filters to input feature maps, enabling the network to detect local visual patterns. Activation functions introduce non-linearity, allowing the network to learn complex relationships. Pooling or other downsampling operations reduce spatial dimensions and computational requirements [3][4].

For image classification, the learned feature maps are passed to a classification head that produces scores or probabilities for the target classes. Although CNNs can be trained from scratch, doing so may require substantial training data and computational resources. Transfer learning offers an alternative by reusing features learned from a large-scale dataset.

2.3. Transfer Learning

Transfer learning involves adapting a model trained on one task or dataset to a related task. Instead of learning every parameter from random initialization, the model reuses representations learned during its original training [1].

In image classification, a pretrained network can serve as a feature extractor. Its original classification head is replaced with a task-specific head, allowing the network to predict categories from the target dataset.

Two common approaches are feature extraction and fine-tuning [5]. During feature extraction, the pretrained backbone is frozen, and only the newly added classification layers are trained. During fine-tuning, selected layers of the pretrained backbone are made trainable so that their learned representations can adapt to the target dataset.

The proposed project combines a pretrained backbone with a custom classification head and selective fine-tuning. This approach allows the model to reuse general visual representations while adapting part of the network to the Food-11 classification task.

2.4. MobileNetV2

MobileNetV2 is a convolutional neural network architecture designed to provide efficient image representations with relatively low computational cost. It is therefore suitable for applications where computational resources and inference efficiency are important considerations.

MobileNetV2 uses depthwise separable convolutions to reduce computational cost, splitting spatial filtering (depthwise) from channel-wise feature combination (pointwise). It also introduces inverted residual blocks with linear bottlenecks, which preserve representational power in the narrow layers while keeping computation and memory low [6].

In this project, MobileNetV2 pretrained on ImageNet is used as the backbone network. The original classification output layer is excluded so that the extracted visual features can be connected to a custom output layer for the eleven Food-11 classes. The pretrained network is selectively fine-tuned during training to adapt its representations to the target dataset.

The following table shows the standard MobileNetV2 architecture [7]:

Layer type	Input size	Output size	Kernel size	Stride	Expansion factor
Initial Conv	224x224x3	112x112x32	3x3	2	-
Inverted Residual Block	112x112x32	112x112x16	3x3	1	1
Inverted Residual Block x2	112x112x16	56x56x24	3x3	2	6
Inverted Residual Block x3	56x56x24	28x28x32	3x3	2	6
Inverted Residual Block x4	28x28x32	14x14x64	3x3	2	6
Inverted Residual Block x3	14x14x64	14x14x96	3x3	1	6
Inverted Residual Block x3	14x14x96	7x7x160	3x3	2	6
Inverted Residual Block x1	7x7x160	7x7x320	3x3	1	6
Final Conv	7x7x320	7x7x1280	1x1	1	-
Global Avg Pooling	7x7x1280	1x1x1280	-	-	-
Fully Connected	1x1x1280	1x1x1000	-	-	-

Table 1 – Standard architecture of MobileNetV2

3. Dataset description and Exploratory Data Analysis (EDA)

3.1. Dataset overview

The Food-11 dataset is used in *The Feast of Divine Judgement* to develop and evaluate a multiclass food image classification model. It consists of food images grouped into eleven predefined categories. Each image is associated with a class label, allowing the model to learn the visual characteristics of different food categories through supervised learning.

This version of the dataset provides separate subsets for training, validation, and evaluation. The training subset is used to learn model parameters, while the validation subset is used to assess model performance during development and select suitable hyperparameter configurations. The evaluation subset is reserved for measuring the performance of the selected model on unseen data.

The use of separate subsets supports a structured experimental workflow and helps prevent the evaluation data from influencing model selection.

3.2. Dataset structure and distribution

The dataset is organized into three directories: training, validation, and evaluation. During dataset inspection, the following image counts were obtained.

Dataset split	Number of images	Purpose
Training	9866	Model training
Validation	3430	Model selection and hyperparameter tuning
Evaluation	3347	Final model evaluation

Table 2 – Image count across the different dataset splits

The training subset contains the largest number of images because it is used to learn the model's parameters. The validation and evaluation subsets provide separate data for model selection and final performance assessment, respectively. The evaluation subset is not used during hyperparameter optimization. This separation ensures that the final test metrics provide an assessment of the selected model on data that was not used to guide model development.

3.3. Food categories

The dataset contains eleven food categories, covering a variety of food types and visual characteristics.

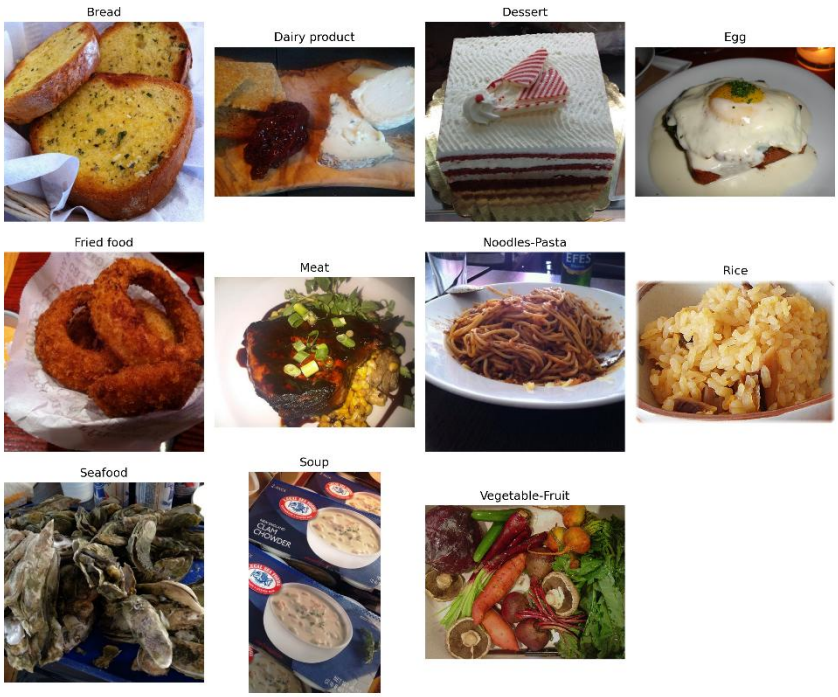


Figure 1 – Sample images of each of the eleven food categories

These categories form the target classes of the classification problem. The model produces a probability for each of the eleven categories and assigns the input image to the category with the highest predicted probability. The categories exhibit different visual characteristics, but some may share similarities in colour, texture, or presentation. Consequently, the model must learn discriminative features that enable it to distinguish between visually similar classes.

3.4. Class distribution analysis

An examination of the training dataset reveals that the number of images is not distributed equally across all eleven categories. This variation represents a class imbalance that may influence the learning process and the model's classification performance.

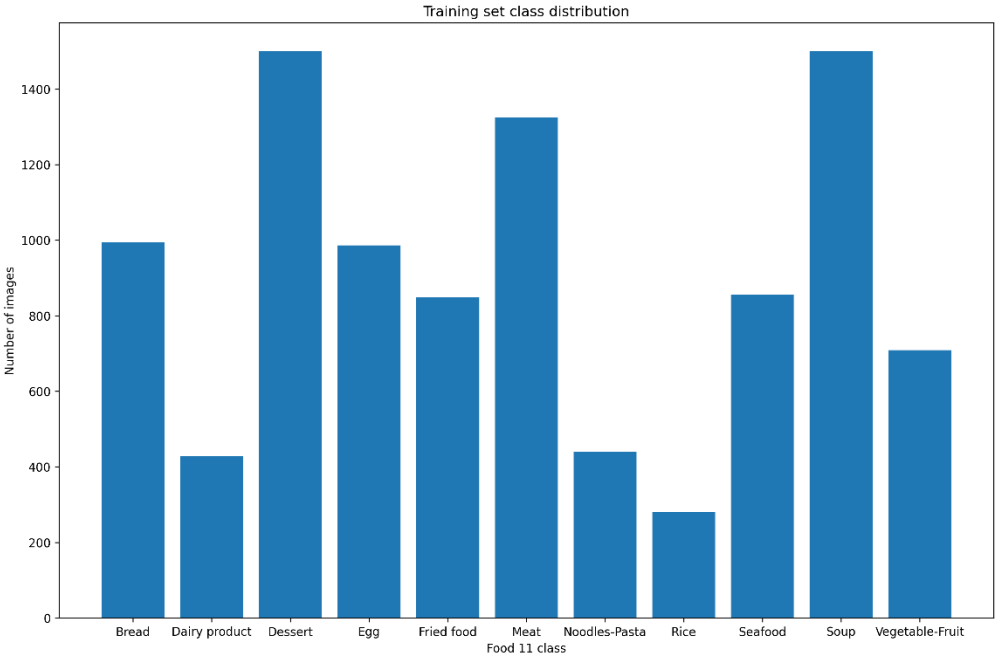


Figure 2 – Training set class distribution

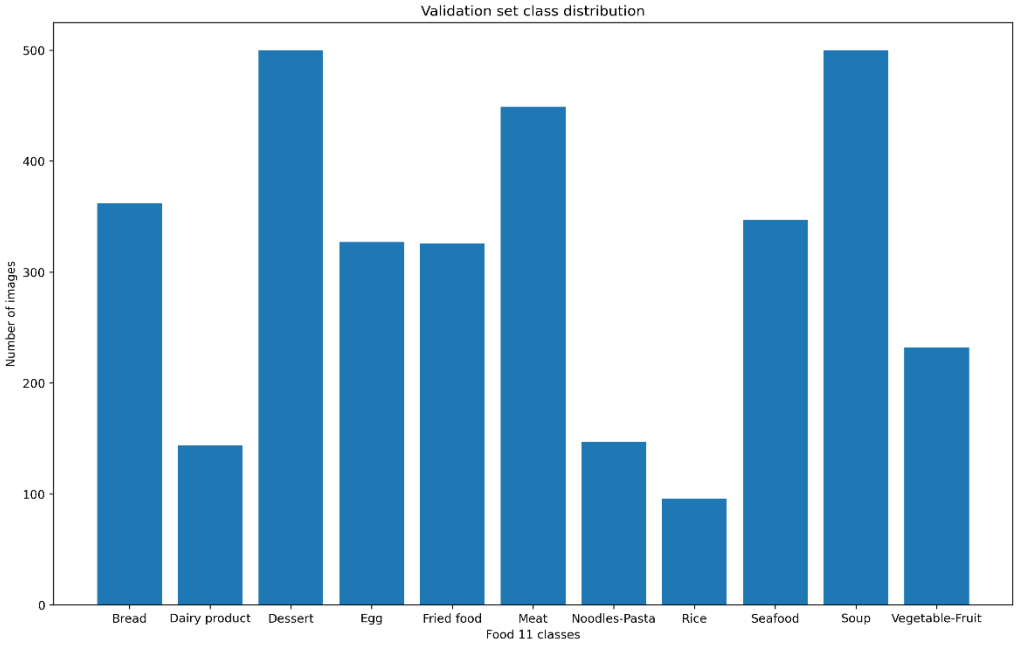


Figure 3 – Validation set class distribution

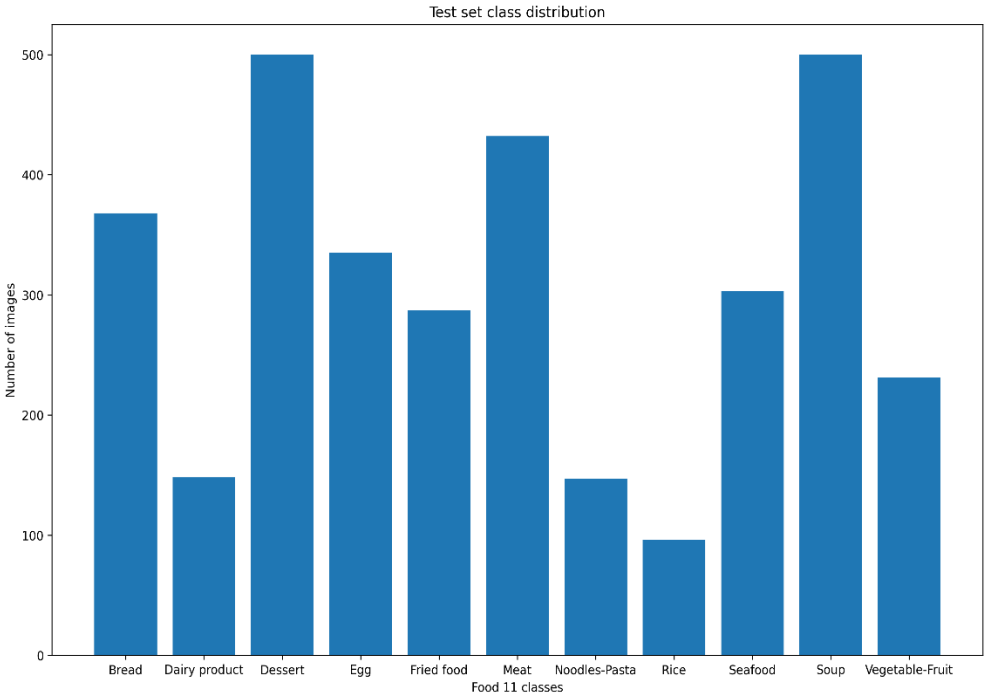


Figure 4 – Test set class distribution

The difference between the least-represented and more frequently represented categories for the training set is substantial. A model trained on such data may learn the characteristics of frequently represented classes more effectively because it encounters more examples of those classes during training. However, the presence of class imbalance does not necessarily imply poor classification performance for minority classes. Its actual effect must be examined using class-wise evaluation metrics and the confusion matrix. The observed imbalance motivates the use of data augmentation during training and macro-averaged evaluation metrics. These measures are discussed in greater detail in the subsequent chapters.

3.5. Exploratory Data Analysis

Exploratory data analysis (EDA) is performed to understand the dataset before model development. It helps identify the dataset's structure, the distribution of class labels, and characteristics of the available image data that may influence the modelling approach.

The EDA process for this project focuses on the following aspects:

1. Dataset size and organization

The number of images in the training, validation, and evaluation subsets is examined to establish the scale of the dataset and verify its organization.

2. Class distribution

The number of training images belonging to each category is examined to identify differences in class representation. The observed imbalance provides useful context for selecting evaluation metrics and designing the training pipeline.

3. Image characteristics

The images represent different food categories, which may vary in shape, colour, texture, and visual composition. Such variation makes it important for the model to learn useful visual features rather than rely on a limited set of superficial characteristics.

4. Implications for model development

The dataset size, diversity of food categories, and uneven class distribution inform subsequent implementation decisions. These include using a pretrained convolutional neural network, applying augmentation to training images, and evaluating performance with metrics that account for all classes.

The EDA findings provide an initial understanding of the data and establish the basis for the preprocessing and modelling decisions described in the following chapters.

3.6. Implications for the proposed model

The characteristics of the Food-11 dataset influence several aspects of the proposed classification system.

First, the eleven-class classification task requires a model capable of extracting meaningful visual features and distinguishing between multiple food categories. MobileNetV2, pretrained on ImageNet, is used to provide a feature extraction foundation without requiring the entire network to be trained from scratch.

Second, the variation in class frequencies highlights the importance of evaluating performance beyond overall accuracy. Macro precision, macro recall, and macro F1-score are therefore included in the evaluation process to give equal weight to the individual classes when calculating their respective averages.

Finally, the separation of the dataset into training, validation, and evaluation subsets supports systematic model development. Hyperparameter configurations are selected using validation performance, while the evaluation subset is reserved for the final assessment of the selected model.

These considerations establish the dataset-related requirements for the preprocessing pipeline and model training methodology presented in the subsequent chapters.

4. Data Preprocessing and Augmentation

4.1. Image preprocessing

The Food-11 images are processed using TensorFlow before being passed to the classification model. Each image is decoded into RGB format and resized to 256 × 256 pixels to ensure a consistent input shape for MobileNetV2. This fixed input resolution is based on the approach described in the research paper *Computer Vision in the Food Industry: Accurate, Real-time, and Automatic Food Recognition with Pretrained MobileNetV2* by Shayan Rokhva, Babak Teimourpour, and Amir Hossein Soltani [9].

OpenCV (cv2) is used for image-reading and image-processing operations where required. The main preprocessing pipeline uses TensorFlow for image decoding, resizing, and conversion into tensors suitable for model training and inference.

Pixel normalization is performed within the model using the Keras Rescaling layer, configured with a scale of 1/127.5 and an offset of -1. The normalization operation is defined as [8]:

$$Normalized\ pixel\ value = \left(\frac{Original\ pixel\ value}{127.5}\right) - 1 \quad (1)$$

The original pixel values range from 0 to 255. This transformation maps them to approximately −1 to 1, matching the expected input representation of the pretrained MobileNetV2 network. Normalization is not performed in the preprocessing function itself; instead, it is applied when the images enter the model.

4.2. Data augmentation

Data augmentation increases the diversity of training samples by applying random transformations to existing images. This helps the model learn useful visual features and reduces its dependence on specific image characteristics. A common seed was also used – seed 42 for reproducibility purposes.

The following augmentation techniques are implemented using Keras layers:

Technique	Configuration	Purpose
Random horizontal flipping	RandomFlip("Horizontal")	Introduces horizontal orientation variations
Random rotation	RandomRotation(0.1)	Varies image orientation
Random brightness	RandomBrightness(0.1)	Simulates brightness variations
Random contrast	RandomContrast(0.1)	Introduces contrast variations
Random zoom	RandomZoom(0.1)	Introduces scale variations
Random erasing	RandomErasing(0.1)	Masks image regions to encourage learning from other features

Table 3 – Image augmentation techniques used in The Feast of Divine Judgement

These transformations introduce variations in orientation, lighting, scale, and visible image regions. Augmentation is applied only to the training dataset. Validation and evaluation images are processed without random augmentation to ensure consistent model assessment.

4.3. Label encoding and data pipeline

The eleven food categories are represented using integer labels ranging from 0 to 10. Each image path is paired with the integer label corresponding to its category. This encoding is compatible with the model's eleven-class softmax output and the sparse categorical cross-entropy loss function used during training. The data pipeline is implemented using TensorFlow's tf.data.Dataset API. Image paths and labels are converted into a dataset, after which each image is read, decoded, resized, and augmented when the training flag is enabled.

The training dataset is shuffled using a fixed seed of 42, with reshuffling enabled between iterations. The Dataset.map() operation processes images using one parallel call for training and four for validation and evaluation, with deterministic execution enabled. The dataset is then divided into batches according to the configured batch size. Finally, prefetch(2) prepares up to two batches ahead of consumption, allowing input preparation to overlap with model computation and potentially reducing waiting time during training.

5. Model architecture and training methodology

5.1. Model architecture

The project uses transfer learning with MobileNetV2 pretrained on ImageNet. The original classification layer is removed, and a custom classification layer is added to classify images into the eleven Food-11 categories.

Component	Configuration	Purpose
Input layer	256 x 256 x 3	Accepts RGB images
MobileNetV2	ImageNet pretrained, include_top=False, selective fine-tuning of the final configurable layers	Extracts visual features while allowing selected pretrained weights to adapt to the Food-11 dataset
Global average pooling	GlobalAveragePooling2D	Converts spatial feature maps into a compact feature vector
Dense output layer	11 neurons, softmax activation	Produces probabilities for the eleven classes

Table 4 – Main components of the model used for The Feast of Divine Judgement

Global average pooling calculates the average activation of each feature map, reducing the spatial dimensions before classification. This produces a compact representation without the large number of parameters that a flattening operation could introduce.

The final dense layer uses softmax activation to produce a probability distribution across the eleven food categories. The category with the highest predicted probability is selected as the model's prediction.

5.2. Transfer learning and fine tuning

MobileNetV2 is initialized with weights learned from ImageNet. These pretrained weights provide visual representations that can be adapted to the Food-11 classification task. The model uses selective fine-tuning rather than training every layer from scratch. Earlier layers of the backbone are frozen, while a configurable number of later layers are allowed to update their weights during training.

Batch normalization layers within the fine-tuned portion remain non-trainable to help maintain training stability. This approach allows selected feature representations to adapt to food images while retaining useful knowledge acquired during pretraining. The number of fine-tuned layers is also considered during hyperparameter optimization.

5.3. Loss function and optimizer

The model is trained using the Adam optimizer and sparse categorical cross-entropy loss. Adam uses adaptive parameter updates based on estimates of the first and second moments of gradients. It is used to optimize the model's trainable parameters during backpropagation.

Sparse categorical cross-entropy is appropriate because the target labels are represented as integer class indices rather than one-hot encoded vectors. It measures the discrepancy between the predicted class probabilities and the correct labels.

Classification accuracy is monitored alongside the loss to track the proportion of correctly classified images during training and validation.

5.4. Regularization and training configuration

Dropout is incorporated into the classification layer to help reduce overfitting by randomly disabling a proportion of activations during training, encouraging the model to learn representations that do not depend excessively on individual features. L2 regularization is applied to the weights of the final dense layer, penalizing large weight values and discouraging unnecessarily large parameters.

The training configuration includes the following parameters:

Parameter	Description
Learning rate	Controls the magnitude of optimizer updates
Batch size	Determines the number of images processed per batch
Number of epochs	Specifies the maximum number of training epochs
Dropout rate	Controls the proportion of activations dropped during training
L2 regularization rate	Controls the regularization penalty on the output layer's weights
Fine tuned layers	Determines how many final MobileNetV2 layers are eligible for training

Table 5 – Parameters in the Feast of Divine Judgement

The appropriate values for these parameters are determined through the randomized hyperparameter optimization procedure described in Chapter 6.

5.5. Hardware and training environment

Model training was performed using an NVIDIA GeForce RTX graphics processing unit (GPU). GPU acceleration enables computationally intensive neural network operations, including convolution and gradient calculations, to be performed more efficiently than relying exclusively on a CPU.

The training implementation uses TensorFlow and Keras. TensorFlow's GPU support relies on compatible NVIDIA GPU drivers and the CUDA and cuDNN software stack where required by the installed TensorFlow environment. CUDA provides GPU computing capabilities, while cuDNN supplies optimized routines for deep learning operation [10][11].

The training script enables GPU memory growth when a GPU is detected, allowing TensorFlow to allocate GPU memory as required rather than immediately reserving all available GPU memory. Mixed-precision computation is enabled through TensorFlow's mixed_float16 policy, allowing supported operations to use 16-bit floating-point computation to potentially improve computational throughput and reduce memory consumption on compatible hardware. The final dense classification layer explicitly produces float32 outputs.

To improve reproducibility, the script sets the random seed to 42 for Python, NumPy, and TensorFlow, and enables deterministic TensorFlow operations. These settings help reduce randomness across experiments, although exact reproducibility may still depend on the hardware and software environment.

5.6. Training procedure and early stopping

The model is trained using the training dataset, with validation data supplied to monitor performance during training. The maximum number of epochs is configurable and is selected as part of the hyperparameter optimization process.

An early stopping callback monitors validation loss with a patience of five epochs. If validation loss does not improve for five consecutive epochs, training stops. The restore_best_weights=True setting restores the model weights from the epoch with the lowest validation loss.

Training and validation accuracy and loss are recorded throughout the process. These values are plotted against the epoch number to examine learning progress and identify potential differences between training and validation performance. The selected model is subsequently evaluated on the held-out evaluation dataset. The results of this evaluation are presented in Chapter 7.

6. Hyperparameter tuning

6.1. Objective of hyperparameter tuning

Hyperparameter tuning was performed to identify a configuration that improved the classification performance of the MobileNetV2 model on the Food-11 dataset. The process explored different combinations of learning rate, batch size, number of epochs, dropout rate, L2 regularization strength, and number of layers selected for fine-tuning.

The macro F1-score on the validation dataset was used as the primary selection metric. This metric gives equal importance to each class by calculating the F1-score separately for every class and taking their unweighted average. It was chosen to assess performance across all eleven food categories, including classes with fewer training examples.

6.2. Hyperparameter search space

The ParameterSampler utility from Scikit-learn was used to generate candidate hyperparameter combinations. A total of 20 combinations were sampled from the predefined search space.

Hyperparameter	Search space
Learning rate	[1e-5, 5e-5, 1e-4, 5e-4]
Batch size	[16, 32]
Number of epochs	[20,25,30]
Dropout rate	[0.1,0.2,0.3,0.4]
L2 regularization strength	[0.00001,0.0001,0.001]
Number of fine-tuned layers	[30, 60, 100]

Table 6 – Hyperparameters and their search spaces

The learning rate controls the magnitude of weight updates during optimization, while the batch size determines the number of training examples processed in each batch. The number of epochs specifies the maximum number of passes through the training dataset. The dropout rate and L2 regularization strength control regularization, whereas the number of fine-tuned layers determines how much of the pretrained MobileNetV2 backbone can adapt to the target dataset.

6.3. Tuning procedure

For each sampled hyperparameter combination, a model was configured and trained using the training dataset. Its classification performance was subsequently evaluated on the validation dataset.

The validation predictions were used to calculate the macro F1-score, which served as the basis for comparing candidate configurations. The configuration achieving the highest validation macro F1-score was selected as the best-performing configuration.

The evaluation dataset was kept separate from the hyperparameter selection process to preserve its role in the final assessment of the model's generalization performance.

6.4. Best hyperparameter configuration

The best-performing configuration identified during the tuning process is presented below.

Hyperparameter	Selected value
Learning rate	5e-05
Batch size	32
Number of epochs	30
Dropout rate	0.3
L2 regularization strength	1e-05
Number of fine-tuned layers	100

Table 7 – Optimal hyperparameter values

This configuration achieved a validation macro F1-score of approximately 0.9070.

The selected configuration combines a relatively small learning rate with fine-tuning of the final 100 MobileNetV2 layers. A dropout rate of 0.3 and an L2 regularization strength of 1e-5 were selected to help control overfitting while allowing the model to learn features relevant to the Food-11 categories.

The batch size of 32 was used to determine the number of images processed per training step, while the maximum training duration was set to 30 epochs. Early stopping remained enabled to monitor validation loss and restore the best-performing model weights.

7. Model evaluation and results

7.1. Training and validation loss

Training and validation loss were monitored throughout the training process to assess the model's learning behavior and identify potential overfitting. Training loss measures prediction error on the training dataset, whereas validation loss measures prediction error on a separate dataset that is not used to update the model weights.

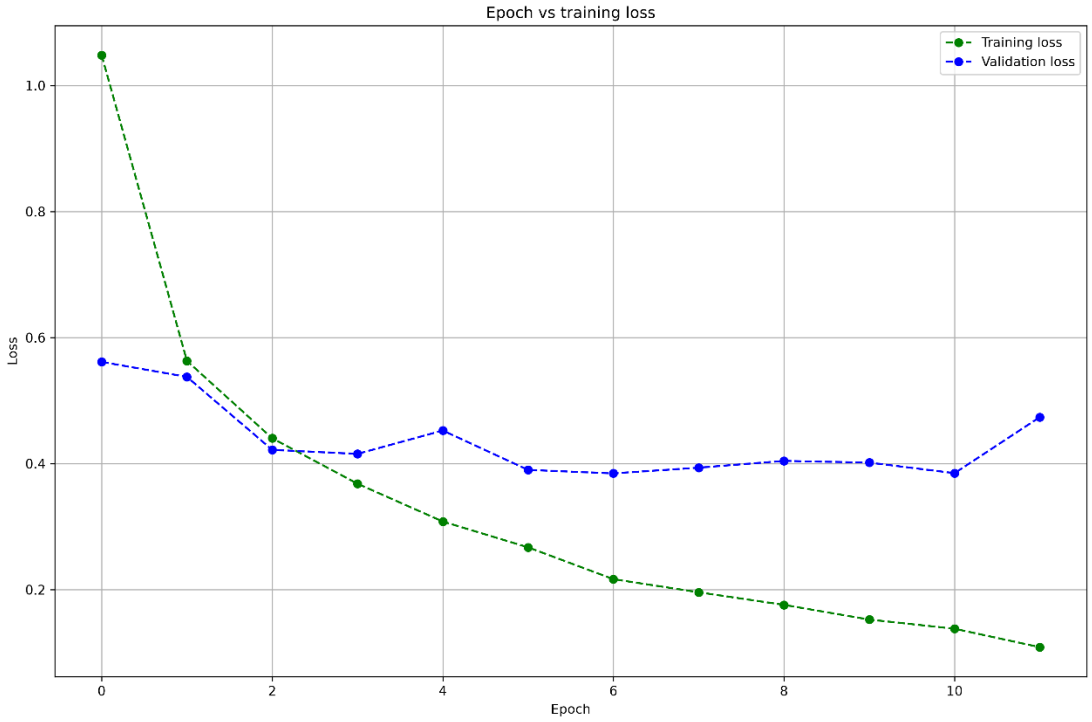


Figure 5 – Training and validation loss across epochs

As shown in Figure 5 training loss decreases consistently from approximately 1.05 in the initial recorded epoch to 0.1 in the final recorded epoch. This indicates that the model progressively improves its fit to the training data. Validation loss initially decreases from approximately 0.56 to 0.42 during the first few epochs. It subsequently fluctuates, reaching its lowest observed value of approximately 0.38 around epoch 10, before increasing to approximately 0.5 in the final recorded epoch.

The divergence between the training and validation loss curves indicates that the model continues improving its fit to the training data while its performance on the validation dataset deteriorates during later training. This behavior suggests overfitting.

The training script uses an early stopping callback that monitors validation loss, with a patience of five epochs and restore_best_weights=True. Training terminates when the callback's stopping condition is met, and the weights from the epoch with the lowest validation loss are restored. Consequently, the graph displays the epochs recorded before early stopping terminated training, rather than the maximum of 30 configured epochs.

7.2. Training and validation accuracy

Training and validation accuracy were monitored alongside loss to assess classification performance throughout the training process.

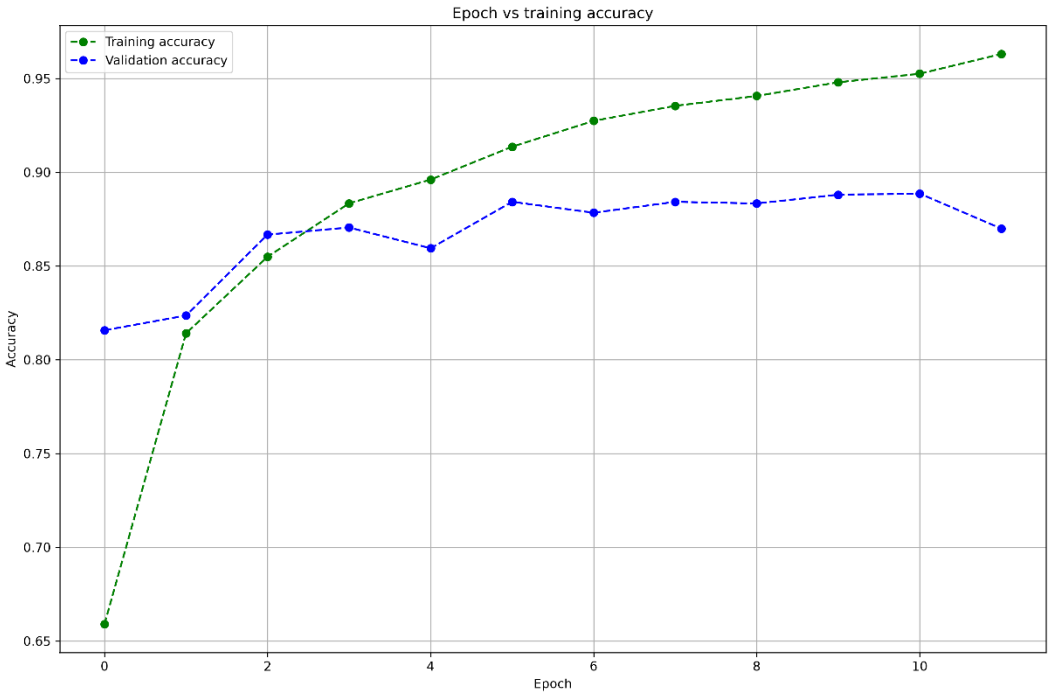


Figure 6 – Training and validation accuracy across epochs

As shown in Figure 6, training accuracy increases from approximately 66% in the initial recorded epoch to 96% in the final recorded epoch. This indicates that the model becomes increasingly effective at classifying images from the training dataset.

Validation accuracy initially improves from approximately 82% to 86% during the first two recorded epochs. It subsequently fluctuates, reaching a maximum of approximately 88% around epoch 10, before declining to approximately 87.0% in the final recorded epoch.

The widening gap between training and validation accuracy provides further evidence of overfitting. Although the model continues improving its training accuracy, these improvements do not translate into equivalent gains on the validation dataset. Considered alongside the loss curves, this suggests that the model's ability to generalize stops improving while its fit to the training data continues to improve.

The accuracy graph covers the same recorded training period as the loss graph. Early stopping is triggered by validation loss, not validation accuracy. Although the maximum training duration was set to 30 epochs, only the epochs recorded before early stopping are displayed.

7.3. Confusion matrix analysis

The confusion matrix provides a detailed view of the model's classification performance across the eleven Food-11 categories. It compares the actual class labels with the predicted labels for the 3,347 images in the held-out evaluation dataset.

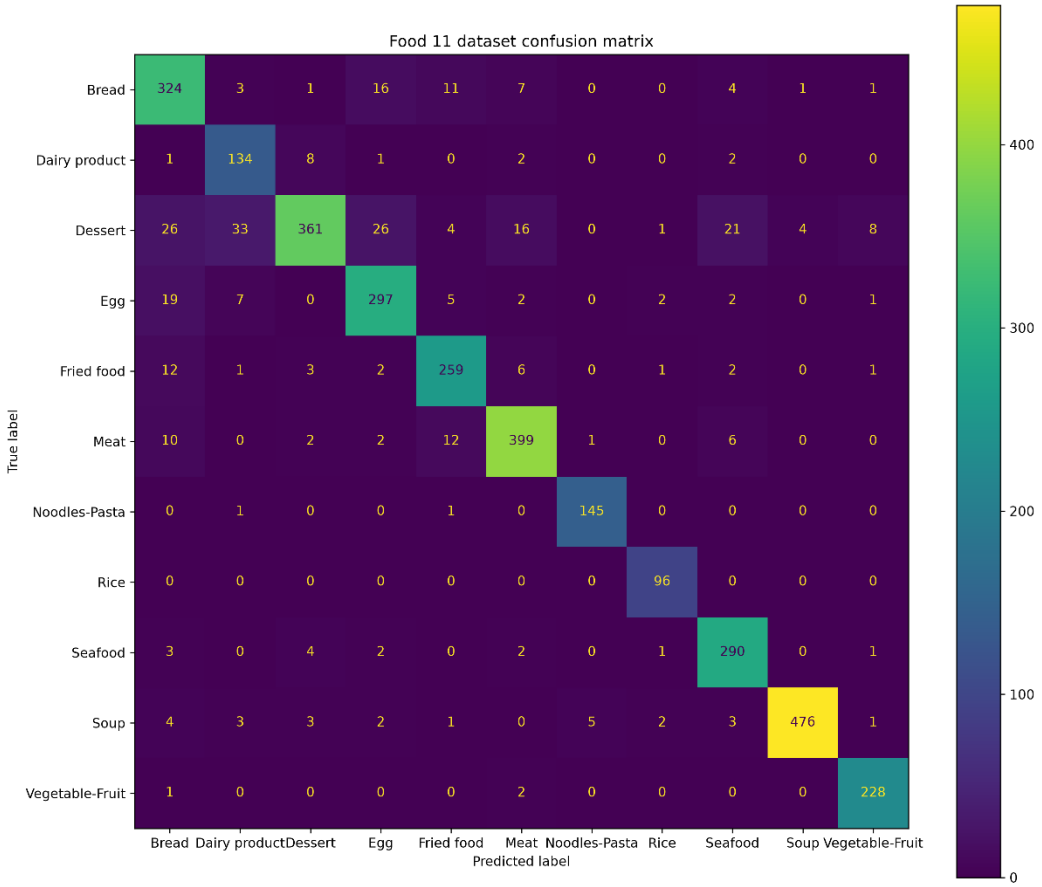


Figure 7 – Confusion matrix for the food 11 classification model

The diagonal entries represent correctly classified images, while the off-diagonal entries represent misclassifications. The concentration of values along the diagonal indicates that the model correctly classifies a substantial proportion of images across the dataset. Several categories demonstrate particularly strong classification performance:

1. Rice: All 96 actual Rice images were correctly classified, corresponding to 100% recall
2. Noodles-Pasta**:** 145 of 147 images were correctly classified
3. Vegetable-Fruit**:** 228 of 231 images were correctly classified
4. Soup**:** 476 of 500 images were correctly classified
5. Meat**:** 399 of 432 images were correctly classified

The confusion matrix also reveals specific classification challenges. Dessert exhibits the largest number of misclassifications, with 139 of its 500 images assigned to other categories. The most frequent errors involve Dairy product (33 images), Bread (26 images), Egg (26 images), Seafood (21 images), and Meat (16 images). These results suggest that the model sometimes struggles to distinguish desserts from visually similar food categories.

Bread is another category with notable misclassifications. Of its 368 images, 324 were correctly classified, while 16 were predicted as Egg and 11 as Fried food. Similarly, 19 Egg images were classified as Bread, indicating some confusion between these categories.

Dairy product also presents a challenge in terms of prediction precision. Although 134 of its 148 actual images were correctly identified, several images from other categories were incorrectly assigned to Dairy product. In particular, 33 Dessert images were classified as Dairy product.

Overall, the confusion matrix indicates that the model performs well on several categories but that its errors are not distributed uniformly across all eleven classes.

7.4. Classification report analysis

The classification report provides precision, recall, F1-score, and support for each food category. These metrics offer a more detailed assessment of the model's performance than overall accuracy alone.

Food category	Precision	Recall	F1 score	Support
Bread	0.8100	0.8804	0.8438	368
Dairy product	0.7363	0.9054	0.8121	148
Dessert	0.9450	0.7220	0.8186	500
Egg	0.8534	0.8866	0.8697	335
Fried food	0.8840	0.9024	0.8931	287
Meat	0.9151	0.9236	0.9194	432
Noodles – Pasta	0.9603	0.9864	0.9732	147
Rice	0.9320	1.0000	0.9648	96
Seafood	0.8788	0.9571	0.9163	303
Soup	0.9896	0.9520	0.9704	500
Vegetable – Fruit	0.9461	0.9870	0.9661	231
Accuracy			0.8990	3347
Macro avg	0.8955	0.9185	0.9043	3347
Weighted avg	0.9037	0.8990	0.8983	3347

Table 8 – Classification report for The Feast of Divine Judgement

Precision measures the proportion of predictions assigned to a class that are correct. Recall measures the proportion of actual instances of a class that the model correctly identifies. The F1-score combines precision and recall, while support indicates the number of actual evaluation samples belonging to each category.

Rice achieves the highest recall at 1.0000, meaning that every actual Rice image in the evaluation dataset was correctly identified. However, its precision is 0.9320 because some images from other categories were also predicted as Rice. This distinction demonstrates why perfect recall does not necessarily imply that every prediction for a class is correct.

Noodles-Pasta and Vegetable-Fruit also achieve strong results, with F1-scores of 0.9732 and 0.9661, respectively. Soup achieves the highest precision at 0.9896, indicating that predictions assigned to this category are highly reliable. Its recall of 0.9520 also indicates that most actual Soup images are correctly identified.

In contrast, Dessert has the lowest recall at 0.7220. Despite achieving high precision of 0.9450, the model fails to identify a substantial proportion of actual Dessert images. This is consistent with the confusion matrix, which shows that many Dessert images are assigned to other categories.

Dairy product has the lowest precision at 0.7363, despite achieving a recall of 0.9054. This indicates that the model identifies most actual Dairy product images but also incorrectly assigns images from other categories to this class. Bread similarly has a comparatively low precision of 0.8100, reflecting the presence of false-positive predictions.

The macro F1-score of 0.9043 represents the unweighted average of the F1-scores across the eleven categories, giving each category equal importance. The weighted F1-score of 0.8983 accounts for the number of actual samples in each category. The difference between these metrics provides additional information about aggregate performance under different class-weighting schemes.

7.5. Overall evaluation results

The final model was evaluated on the held-out evaluation dataset containing 3,347 images. This dataset was kept separate from the training and hyperparameter-selection processes to assess the model's performance on previously unseen samples. The overall test evaluation results are summarized below:

Evaluation metric	Score
Test loss	0.3316
Test accuracy	0.8990
Test macro precision	0.8955
Test macro recall	0.9185
Test macro F1	0.9043

Table 9 – Evaluation metrics of the test set

The model achieves an accuracy of 89.90%, meaning that 3,009 of the 3,347 evaluation images were classified correctly. The macro F1-score of 90.43% indicates strong aggregate performance when all eleven categories are given equal weight.

During hyperparameter tuning, the best configuration achieved a validation macro F1-score of approximately 0.9070. The final evaluation macro F1-score of 0.9043 differs from this value by approximately 0.27 percentage points. This indicates that the final evaluation score is close to the best validation score observed during tuning.

This is consistent with the effect of early stopping. Because restore_best_weights=True, the evaluated model uses the weights from the epoch with the lowest validation loss (around epoch 10), before the divergence visible in the later epochs of the training curves. The overfitting seen in Figures 5 and 6 therefore describes what would have happened had training continued, and it is largely, though not necessarily completely, mitigated in the final model. The held-out evaluation metrics provide a separate assessment of this model’s performance.”

Taken together, the evaluation results indicate that the model performs particularly well on categories such as Rice, Noodles-Pasta, Soup, and Vegetable-Fruit, while Dessert and Dairy product present comparatively greater classification challenges

8. System implementation and user interface

8.1. System overview

The project extends the trained Food-11 image classification model into an interactive application that allows users to obtain food category predictions by uploading images. The application combines a deep learning model with a Streamlit-based user interface and a MongoDB database containing information about the food categories.

The system consists of three principal components: the trained MobileNetV2 classification model, the Streamlit application responsible for user interaction, and the MongoDB database used to retrieve information associated with the predicted food category.

8.2. Streamlit user interface

Streamlit was used to develop the application's user interface in Python. It provides an interface through which users can upload food images and interact with the classification model without directly accessing the underlying implementation.

As a fan of *Genshin Impact*, I decided to incorporate a Genshin-inspired theme into the application's visual design. This personal influence shaped the interface's aesthetic, giving the project a distinctive identity while maintaining its primary focus on food image classification. The theme also allowed me to personalize the application beyond a conventional machine learning demonstration interface.

After an image is uploaded, the application displays the image alongside the predicted food category and its corresponding confidence score. The interface also provides an option to view the predicted probabilities for the individual Food-11 categories, allowing users to examine the model's output beyond the highest-ranked prediction.

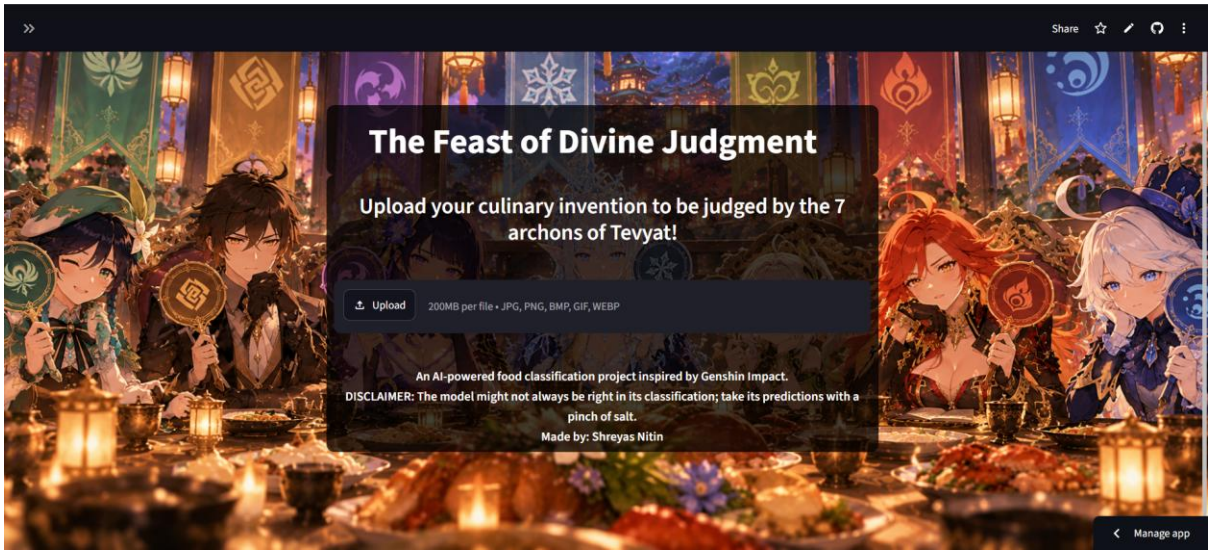


Figure 8 – Genshin Impact inspired UI

The interface is designed to make the classification system accessible through a straightforward workflow. The Genshin-inspired styling provides a personalized visual identity, while the prediction and probability displays keep the application's core functionality central to the user experience.

8.3. Image classification workflow

The application's image classification process follows a sequence of operations:

1. The user uploads a food image through the Streamlit interface.
2. The uploaded image is prepared for inference using the required image dimensions and preprocessing operations.
3. The trained MobileNetV2 model generates probability scores for the eleven Food-11 categories.
4. The category with the highest predicted probability is selected as the predicted class.
5. The interface displays the uploaded image, predicted category, and corresponding confidence score.
6. If requested, the user can view the probability scores associated with the individual food categories.
7. The application retrieves the information associated with the predicted category from MongoDB and displays it in the interface.

This workflow connects image classification, probability visualization, and database retrieval within a single application.

8.4. MongoDB integration

MongoDB was incorporated to store information associated with the eleven food categories. The application uses the PyMongo library to communicate with the database.

The database is named food11, and the collection containing food category information is named foods. Each food category is represented by a document containing information that can be retrieved by the application.

After generating a prediction, the application uses the predicted category to retrieve the corresponding document from MongoDB. The retrieved information is then presented alongside the classification result, providing users with additional details associated with the predicted food category.

MongoDB complements the classification model by separating prediction generation from information retrieval. The model determines the predicted category, while the database supplies the corresponding stored information.

8.5. Integration of the model, interface and database

The application combines image classification, probability display, and database retrieval into a unified workflow. Streamlit manages user interaction, the trained MobileNetV2 model generates predictions, and MongoDB supplies information associated with the predicted category.

The option to view class probabilities provides additional insight into the model's output by allowing users to examine the predicted probability distribution across all eleven categories rather than viewing only the highest-probability result. These probabilities represent the model's predictions and should not be interpreted as guarantees of correctness.

The separation of responsibilities allows the classification model, database, and user interface to perform distinct functions while working together as a single application. The Genshin Impact-inspired design adds a personalized visual identity without changing the underlying classification methodology.

9. Limitations and future work

9.1. Limitations of the system

Although the developed Food-11 classification system achieved an accuracy of 89.90% and a macro F1-score of 90.43% on the held-out evaluation dataset, several limitations were identified during the evaluation process.

9.1.1. Uneven classification performance across categories

The classification report indicates that the model does not perform equally well across all eleven food categories. While categories such as Rice, Noodles-Pasta, Soup, and Vegetable-Fruit achieved strong F1-scores, Dessert and Dairy product presented comparatively greater classification challenges.

Dessert achieved a recall of 72.20%, the lowest among the eleven categories. The confusion matrix showed that several Dessert images were misclassified as Dairy product, Bread, Egg, and Seafood. This suggests that the model sometimes struggles to distinguish visually similar food categories.

Dairy product achieved a precision of 73.63%, the lowest among the classes, despite a recall of 90.54%. This indicates that the model identifies most actual Dairy product images but also incorrectly assigns images from other categories to this class.

These results demonstrate that strong overall performance does not necessarily imply equally reliable predictions for every category.

9.1.2. Evidence of overfitting

The training and validation curves indicate that overfitting occurred during the later stages of training. Training accuracy increased to approximately 96%, while validation accuracy peaked at approximately 88% before declining. Similarly, training loss continued to decrease, whereas validation loss increased after reaching its lowest observed value.

This divergence suggests that continued training improved the model's fit to the training data without producing corresponding improvements in validation performance.

Early stopping was incorporated to address this issue by monitoring validation loss and restoring the model weights from the epoch with the best validation loss. Since the final model uses the best-validation-loss weights, the overfitting observed in the curves reflects later training epochs rather than the evaluated model. Nevertheless, the observed learning behavior indicates that further improvements to regularization and training configuration could be investigated.

9.1.3. Dependence on the Food 11 dataset

The model was trained and evaluated using the Food-11 dataset, which contains eleven predefined food categories. Consequently, its classification capability is limited to these categories and the visual characteristics represented in the dataset. The evaluation results do not establish how well the model would perform on food categories outside Food-11 or on images collected under substantially different conditions. Its performance on such images would require additional testing.

9.1.4. Limitations of the classification approach

The system selects the category with the highest predicted probability from the eleven available classes. Although the interface also allows users to inspect the probability distribution, these values do not guarantee that the prediction is correct. The current classification setup is designed to choose among the predefined categories. Therefore, an image belonging to an unrepresented category may still receive a prediction corresponding to one of the eleven supported classes.

9.2. Future work

Several improvements could be explored to address the limitations identified during development and evaluation.

9.2.1. Improving class specific performance

Future experiments could focus on improving performance for categories with comparatively weaker results, particularly Dessert and Dairy product. Possible approaches include targeted data augmentation, further investigation of class imbalance, and experiments with class-weighted loss functions. These approaches could be evaluated using validation macro F1-score and per-class precision, recall, and F1-score to determine whether improvements in weaker categories can be achieved without substantially reducing performance elsewhere.

9.2.2. Reducing overfitting

Further experiments could investigate alternative regularization configurations, dropout rates, L2 regularization strengths, and fine-tuning strategies. The number of trainable MobileNetV2 layers could also be varied to examine how the extent of fine-tuning affects generalization. Early stopping would remain useful for monitoring validation performance and avoiding unnecessary training once improvements have stalled.

9.2.3. Exploring alternate model architectures

Although MobileNetV2 provided a suitable transfer-learning backbone for the project, future work could compare it with alternative convolutional neural network architectures. Such comparisons could examine classification performance, computational requirements, training time, and model size. Macro F1-score would remain an important evaluation metric to ensure that performance across all food categories is considered.

10. Conclusion

The project, *The Feast of Divine Judgement*, focused on developing a food image classification system using transfer learning and deep learning techniques. The objective was to build a model capable of classifying food images into the eleven categories defined in the Food-11 dataset and integrate the trained model into an interactive application.

MobileNetV2, pretrained on ImageNet, was used as the model backbone, with selective fine-tuning applied to adapt its weights to the Food-11 dataset. Hyperparameter tuning was performed to identify a suitable configuration, while dropout and L2 regularization were incorporated to help control overfitting. Training and validation curves were also examined to assess the model's learning behavior.

The final model achieved an accuracy of 89.90**%** and a macro F1-score of **90.43%** on the held-out evaluation dataset containing 3,347 images. The confusion matrix and classification report provided further insight into class-wise performance, revealing strong results for categories such as Rice, Noodles-Pasta, Soup, and Vegetable-Fruit, alongside greater classification challenges for Dessert and Dairy product.

Beyond model development, the project integrated the trained classifier into a Streamlit application. Users can upload food images, view the predicted category and confidence score, inspect class probabilities, and retrieve associated category information from MongoDB. Inspired by my interest in *Genshin Impact*, the application also incorporates a Genshin-inspired visual theme, giving the project a distinctive identity.

Overall, the project demonstrates the practical application of transfer learning to food image classification and the integration of a trained deep learning model into an interactive software system. It also highlights the importance of evaluating class-wise performance and monitoring overfitting rather than relying exclusively on overall accuracy. The implementation provides a foundation for further improvements through alternative model architectures, refined regularization strategies, expanded dataset coverage, and targeted approaches to improving performance on challenging food categories.

References

1. Murel, J., & Kavlakoglu, E. (2024, February 12). *What is transfer learning?* IBM. https://www.ibm.com/think/topics/transfer-learning
2. Trolukovich, A. (n.d.). *Food-11 image dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/trolukovich/food11-image-dataset
3. IBM. (2021, October 6). *What are convolutional neural networks?* https://www.ibm.com/think/topics/convolutional-neural-networks
4. Amidi, A., & Amidi, S. (n.d.). *CS 230 – Convolutional neural networks cheatsheet*. Stanford University https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/
5. TensorFlow. (n.d.). *Transfer learning and fine-tuning*. https://www.tensorflow.org/tutorials/images/transfer_learning
6. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). *MobileNetV2: Inverted residuals and linear bottlenecks* [Preprint]. arXiv. https://arxiv.org/pdf/1801.04381
7. GeeksforGeeks. (n.d.). *MobileNet V2 architecture in computer vision*. https://www.geeksforgeeks.org/computer-vision/mobilenet-v2-architecture-in-computer-vision/
8.Keras. (n.d.). *Rescaling layer*. https://keras.io/api/layers/preprocessing_layers/image_preprocessing/rescaling/
9. Rokhva, S., Teimourpour, B., & Soltani, A. H. (2024). *Computer vision in the food industry: Accurate, real-time, and automatic food recognition with pretrained MobileNetV2* [Preprint]. arXiv. https://arxiv.org/abs/2405.11621
https://keras.io/api/layers/preprocessing_layers/image_preprocessing/rescaling/
10. Jain, A. (2024, December 15). *What is CUDA and CUDA CNN?* Medium. https://medium.com/@abhishekjainindore24/what-is-cuda-and-cuda-cnn-be03e26235c1
11. NVIDIA. (n.d.). *NVIDIA cuDNN*. NVIDIA Developer. https://developer.nvidia.com/cuda/cuda-x-libraries/cudnn